

Teach Module 19 README

Module 19: Integration Testing and UI Test Automation

This README is based on the Module 19 topic map from `docs/Java Software Engineer bootcamp.docx`, but the teaching content below is written independently and does not reuse the course material.

Learning Goals

By the end of this module, you should be able to:

- Explain the difference between unit tests, integration tests, and UI automation tests.
- Use Spring Boot testing tools such as `@WebMvcTest`, `@DataJpaTest`, and `@SpringBootTest`.
- Apply the test pyramid to decide what kind of tests to write.
- Manage predictable test data for integration tests.
- Write Selenium WebDriver tests for browser-based user workflows.
- Use reliable UI selectors and explicit waits.
- Decide which tests should run in a CI pipeline.

1. Unit Tests vs Integration Tests vs UI Tests

Testing usually happens in layers.

```
Fastest / most isolated
Unit tests
Integration tests
End-to-end UI tests
Slowest / most realistic
```

Unit Tests

A unit test checks one small piece of logic in isolation.

Example question:

```
Does this price calculator return the right discount?
```

Example:

```
@Test
void calculatesTotalWithTax() {
    OrderService service = new OrderService();

    BigDecimal total = service.calculateTotal(new BigDecimal("100"));

    assertEquals(new BigDecimal("108.00"), total);
}
```

This test does not need a database, web server, Spring context, or browser.

Integration Tests

An integration test checks whether multiple parts of the application work together.

Example question:

Does my controller call the service, persist data, and return the expected HTTP response?

Example:

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class OrderControllerIntegrationTest {

    @Autowired
    TestRestTemplate restTemplate;

    @Test
    void createsOrder() {
        OrderRequest request = new OrderRequest("Laptop", 1);

        ResponseEntity<OrderResponse> response =
            restTemplate.postForEntity("/orders", request, OrderResponse.class);

        assertEquals(HttpStatus.CREATED, response.getStatusCode());
        assertEquals("Laptop", response.getBody().itemName());
    }
}
```

This test can verify routing, JSON conversion, validation, controller logic, service wiring, and persistence behavior.

UI Automation Tests

A UI automation test checks the application the way a user experiences it.

Example question:

Can a user open the React app, type into a form, click submit, and see a success message?

UI tests are valuable because they test real workflows, but they are usually slower and more fragile than lower-level tests.

2. Spring Boot Test Slices

Spring Boot gives you different test tools so you do not always have to start the entire application.

@WebMvcTest

Use `@WebMvcTest` when you want to test the controller/web layer.

```
@WebMvcTest(OrderController.class)
class OrderControllerTest {
```

```

@Autowired
MockMvc mockMvc;

@MockBean
OrderService orderService;

@Test
void returnsOrderById() throws Exception {
    when(orderService.findById(1L))
        .thenReturn(new OrderResponse(1L, "Keyboard"));

    mockMvc.perform(get("/orders/1"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.itemName").value("Keyboard"));
}
}

```

Use this when you want to test HTTP routing, JSON responses, validation, and controller behavior without loading the whole application.

@DataJpaTest

Use `@DataJpaTest` when you want to test repository and database mapping behavior.

```

@DataJpaTest
class OrderRepositoryTest {

    @Autowired
    OrderRepository repository;

    @Test
    void savesOrder() {
        Order order = new Order("Mouse", 2);

        Order saved = repository.save(order);

        assertNotNull(saved.getId());
    }
}

```

Use this when you want to test JPA mappings, repository queries, and persistence behavior.

@SpringBootTest

Use `@SpringBootTest` when you want the full Spring application context.

```

@SpringBootTest
class CustomerServiceIntegrationTest {

    @Autowired

```

```

    CustomerService customerService;

    @Autowired
    CustomerRepository customerRepository;

    @Test
    void registersCustomer() {
        Customer customer = customerService.register("Ava", "ava@example.com");

        assertTrue(customerRepository.findByEmail("ava@example.com").isPresent());
    }
}

```

Rule of thumb:

Use the smallest test setup that gives you the confidence you need.

3. The Test Pyramid

The test pyramid is a strategy for balancing speed, reliability, and confidence.

Many unit tests
 Some integration tests
 Few UI/end-to-end tests

Why?

- Unit tests are fast and cheap.
- Integration tests prove that real application pieces work together.
- UI tests prove that important user journeys work, but they take longer and break more easily when the UI changes.

A healthy project might look like this:

70% unit tests
 20% integration tests
 10% UI/end-to-end tests

Good UI automation candidates:

Login
 Create account
 Submit important form
 Search/filter workflow
 Checkout/payment flow
 Admin approval flow

Do not automate every possible browser path. Focus UI automation on the workflows that would seriously hurt if they broke.

4. Test Data Management

Integration tests need predictable data.

Bad test data strategy:

```
Assume the database already has user ID 5
Assume another test did not modify it
Assume tests always run in the same order
```

Better strategy:

```
@BeforeEach
void setUp() {
    repository.deleteAll();
    repository.save(new Customer("Ava", "ava@example.com"));
}
```

Common approaches:

```
H2 for lightweight database tests
Testcontainers for realistic database tests
SQL scripts for setup data
@Transactional rollback for cleanup
```

Example:

```
@Test
@Transactional
void findsCustomerByEmail() {
    Customer customer = new Customer("Sam", "sam@example.com");
    repository.save(customer);

    Optional<Customer> found = repository.findByEmail("sam@example.com");

    assertTrue(found.isPresent());
}
```

Good integration tests should be independent. One test should not depend on another test running first.

5. Selenium WebDriver for UI Testing

Selenium controls a real browser.

A Selenium test usually does four things:

```
Open a page
Find elements
```

```
Perform user actions
Assert visible results
```

Example:

```
WebDriver driver = new ChromeDriver();

driver.get("http://localhost:3000/login");

driver.findElement(By.id("username")).sendKeys("admin");
driver.findElement(By.id("password")).sendKeys("password");
driver.findElement(By.cssSelector("button[type='submit']")).click();

WebElement heading = driver.findElement(By.tagName("h1"));

assertEquals("Dashboard", heading.getText());

driver.quit();
```

6. Reliable Selectors

The fragile part of Selenium testing is often element selection.

Avoid selectors like this:

```
By.cssSelector("div > div > div:nth-child(2) > button")
```

Prefer stable selectors:

```
<button data-testid="login-submit">Sign in</button>
```

Then test with:

```
driver.findElement(By.cssSelector("[data-testid='login-submit']")).click();
```

For React applications, stable selectors are especially helpful because the component structure can change often.

7. Waiting for React UI

Modern frontends are asynchronous. Do not assume an element appears instantly.

Bad:

```
Thread.sleep(3000);
```

Better:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
```

```
WebElement message = wait.until(  
    ExpectedConditions.visibilityOfElementLocated(  
        By.cssSelector("[data-testid='success-message']")  
    )  
);  
  
assertEquals("Order created", message.getText());
```

Selenium tests should wait for specific conditions, not fixed time.

8. Running Tests in CI

In a CI pipeline, split tests by speed and purpose.

```
Unit tests: every commit  
Integration tests: every pull request  
Selenium smoke tests: before deployment  
Full UI suite: nightly or scheduled run
```

A typical Maven command:

```
mvn test
```

This usually runs unit tests.

Another common command:

```
mvn verify
```

This often runs integration tests too, depending on plugin setup.

For UI tests in CI, you usually need:

```
Application backend running  
Frontend running  
Browser driver available  
Headless browser mode  
Test credentials or test data
```

Headless browser example:

```
ChromeOptions options = new ChromeOptions();  
options.addArguments("--headless=new");  
  
WebDriver driver = new ChromeDriver(options);
```

What to Remember

Integration testing answers:

Do our application pieces work together?

UI automation answers:

Can a real user complete the workflow?

Practical checklist:

- Know the difference between unit and integration tests
- Use Spring Boot test slices correctly
- Understand the test pyramid
- Control test data deliberately
- Write Selenium tests with reliable selectors
- Use waits instead of sleeps
- Run tests sensibly in CI

Practice Exercises

Exercise 1: Controller Integration Test

Create a simple Spring Boot endpoint:

```
POST /customers
GET /customers/{id}
```

Write an integration test that:

- Sends a JSON request
- Verifies HTTP 201 Created
- Fetches the saved customer
- Checks response fields

Focus: `@SpringBootTest` , `TestRestTemplate` , request/response validation.

Exercise 2: MockMvc API Test

Write tests for a controller using `MockMvc` .

Test cases:

```
GET /customers/1 returns 200
GET /customers/999 returns 404
POST /customers with missing email returns 400
POST /customers with valid data returns 201
```

Focus: HTTP status codes, JSON assertions, validation errors.

Exercise 3: Repository Integration Test

Create a `CustomerRepository` .

Practice:

```
Save a customer
Find customer by email
Update customer name
Delete customer
Verify empty result after delete
```

Focus: `@DataJpaTest` , database mappings, predictable test data.

Exercise 4: Service and Repository Integration Test

Create a service method:

```
Customer registerCustomer(CustomerRequest request)
```

Test that it:

```
Saves valid customers
Rejects duplicate emails
Throws a meaningful exception
Persists correct data
```

Focus: real service and real repository working together.

Exercise 5: Test Data Cleanup

Write integration tests that use setup and cleanup.

```
@BeforeEach
void setUp() {
    repository.deleteAll();
}
```

Create multiple tests that each insert their own data.

Focus: making tests independent from each other.

Exercise 6: Test Pyramid Classification

Classify each test idea as unit, integration, or UI:

```
Password validator rejects weak password
POST /orders stores order in database
User clicks Login and sees Dashboard
Repository finds account by username
Service calculates discount
Checkout page submits payment form
```

Focus: knowing which test layer fits which problem.

Exercise 7: Selenium Login Test

Build or use a simple login page.

Automate this flow:

```
Open login page
Enter username
Enter password
Click Sign In
Verify Dashboard heading appears
```

Focus: Selenium WebDriver basics.

Exercise 8: Selenium Form Submission Test

Create a React form for adding a customer.

Automate:

```
Type name
Type email
Click submit
Wait for success message
Verify success message text
```

Focus: form input, button clicks, explicit waits.

Exercise 9: Reliable Selectors

Add stable selectors to React elements:

```
<input data-testid="customer-name" />
<input data-testid="customer-email" />
<button data-testid="submit-customer">Save</button>
```

Then use Selenium selectors:

```
By.cssSelector("[data-testid='customer-email']")
```

Focus: avoiding fragile selectors like `nth-child` .

Exercise 10: Replace Sleep with Wait

Start with a bad Selenium test using:

```
Thread.sleep(3000);
```

Then refactor it to:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
```

Focus: reliable asynchronous UI testing.

Exercise 11: Full End-to-End Customer Flow

Combine backend and frontend.

Scenario:

```
Start backend
Start frontend
Open React page
Create customer through UI
Verify success message
Call backend API or database to confirm customer exists
```

Focus: complete user journey across UI, API, service, and persistence.

Exercise 12: CI Testing Plan

Write a short test execution plan:

```
Which tests should run on every commit?
Which tests should run on every pull request?
Which tests should run before deployment?
Which tests are too slow for every commit?
```

Example:

```
Unit tests: every commit
Integration tests: every pull request
Selenium smoke test: before deployment
Full Selenium suite: nightly
```

Recommended Practice Sequence

1. MockMvc controller tests
2. @DataJpaTest repository tests
3. @SpringBootTest full integration tests
4. Selenium form automation
5. CI testing strategy

Module 19 Lab: Integration and UI Testing

Goal

Build a small customer feature and test it at the API, database, and UI levels.

Lab Scenario

You are building a simple customer registration feature.

A user should be able to:

```
Enter customer name
Enter customer email
Submit the form
See a success message
Verify the customer is saved
```

Part 1: Backend Setup

Create a Spring Boot feature with:

```
Customer entity
CustomerRepository
CustomerService
CustomerController
```

Example endpoints:

```
POST /api/customers
GET /api/customers/{id}
GET /api/customers/email/{email}
```

Customer fields:

```
id
name
email
```

Validation rules:

```
Name is required
Email is required
Email must be valid
Duplicate email is not allowed
```

Part 2: Repository Integration Test

Write a `@DataJpaTest` .

Test cases:

```
Save customer successfully
Find customer by email
```

```
Return empty result for unknown email
Delete customer successfully
```

Part 3: Controller Integration Test

Write a `@SpringBootTest` or `@WebMvcTest` with `MockMvc` .

Test cases:

```
POST valid customer returns 201
POST missing name returns 400
POST invalid email returns 400
GET existing customer returns 200
GET unknown customer returns 404
```

Example assertion style:

```
mockMvc.perform(post("/api/customers")
    .contentType(MediaType.APPLICATION_JSON)
    .content("""
        {
            "name": "Ava Patel",
            "email": "ava@example.com"
        }
    """))
    .andExpect(status().isCreated())
    .andExpect(jsonPath("$.name").value("Ava Patel"))
    .andExpect(jsonPath("$.email").value("ava@example.com"));
```

Part 4: React UI

Create a simple page with:

```
Name input
Email input
Submit button
Success message
Error message
```

Use stable test selectors:

```
<input data-testid="customer-name" />
<input data-testid="customer-email" />
<button data-testid="save-customer">Save</button>
<div data-testid="success-message">Customer saved</div>
```

Part 5: Selenium UI Test

Automate this flow:

```
Open customer form page
Type name
Type email
Click Save
Wait for success message
Verify message text
```

Example Selenium pattern:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

driver.findElement(By.cssSelector("[data-testid='customer-name']"))
    .sendKeys("Ava Patel");

driver.findElement(By.cssSelector("[data-testid='customer-email']"))
    .sendKeys("ava@example.com");

driver.findElement(By.cssSelector("[data-testid='save-customer']"))
    .click();

WebElement message = wait.until(
    ExpectedConditions.visibilityOfElementLocated(
        By.cssSelector("[data-testid='success-message']")
    )
);

assertEquals("Customer saved", message.getText());
```

Part 6: CI Plan

Write a short plan answering:

```
Which tests should run on every commit?
Which tests should run on pull requests?
Which tests should run before deployment?
Which tests are too slow for every commit?
```

Expected idea:

```
Unit tests: every commit
Repository/controller integration tests: pull request
Selenium smoke test: before deployment
Full Selenium suite: nightly
```

Lab Deliverables

Submit:

```
Customer backend code
Repository integration tests
Controller/API integration tests
React customer form
Selenium UI test
Short CI test strategy
```

Success Criteria

You are done when:

```
Backend tests pass
Invalid input is rejected
Customer can be created through the UI
Selenium test verifies the UI flow
Test data does not depend on test order
No Thread.sleep is used in Selenium tests
```